# 位并行与向量化指令集在真值表生成中的应用

熊洋铭

2026/3/24

## 输入

((!p&q)-(p|q))>r

- 一个任意的 wff。
- 字符集包含：
  - 小写字母代表的命题变元 pqr
  - 逻辑运算符 !&|>-
  - 括号 ( )
- wff 长度 $m$。
- 不同命题变元个数 $n$。

## 算法：生成 wff 的真值表

`generate_truth_table_from_wff()`

## 输出

p 00001111
q 00110011
r 01010101
* 01011111

- 输入 wff 对应的真值表
- 按顺序列举了各命题变元取值和命题取值
- 大小为 $(n+1)\times 2^n$

# 算法：生成 wff 的真值表

generate_truth_table_from_wff()

输入

**提取命题变元** `(!p|q)&r` ----------> `[p, q, r]`

**转为后缀表达式** `(!p|q)&r` ----------> `[p,!,q,|,r,&]`

**取值并计算** `[p,!,q,|,r,&]` `for S in [0, 1 << n)` ----------> `truth table`

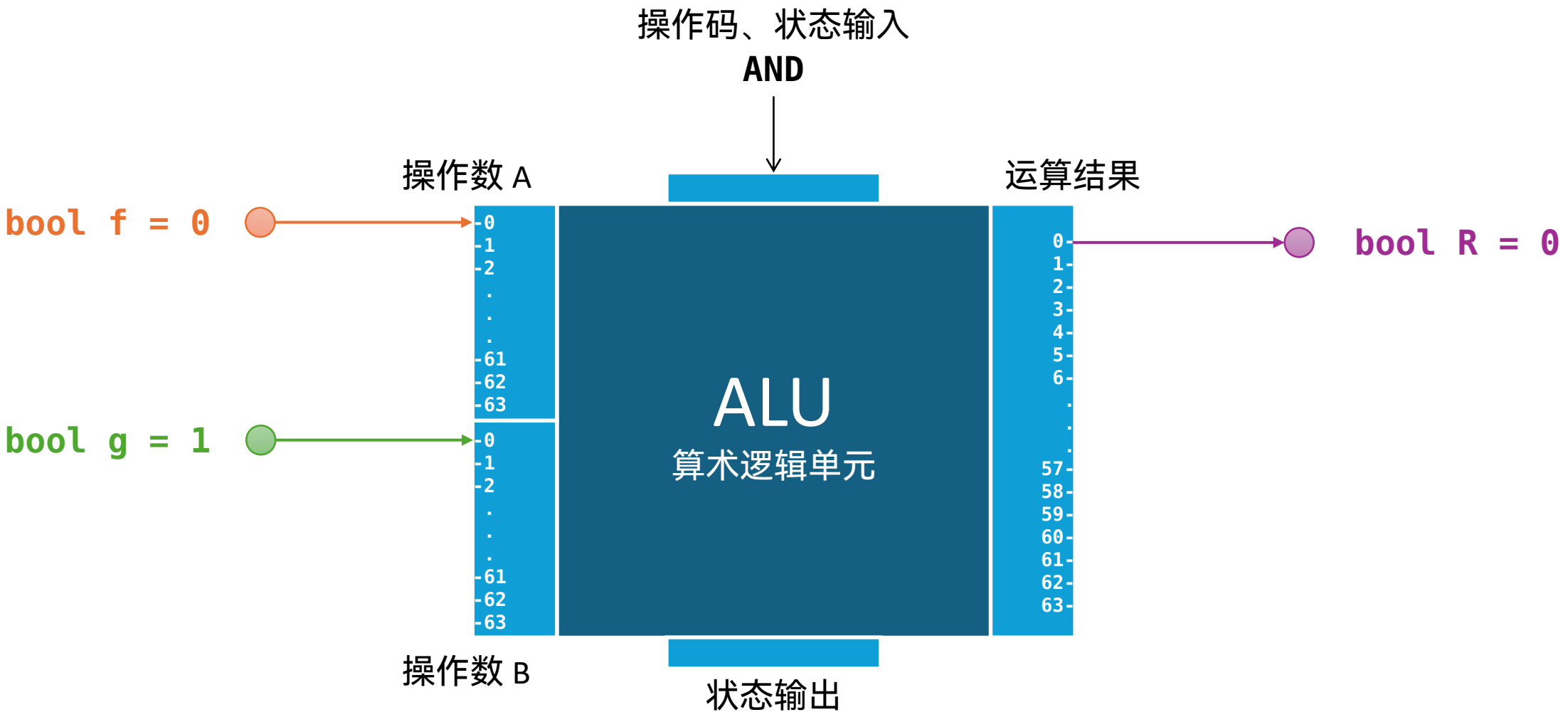
输出

| 过程 | 图示 | | | 时间复杂度 |
|---|---|---|---|---|
| **提取命题变元** | `(!p\|q)&r` | ----------> | `[p, q, r]` | $O\left(m + n\log n\right)$ |
| **转为后缀表达式** | `(!p\|q)&r` | -------> | `[p,!,q,\|,r,&]` | $O\left(m\right)$ |
| **取值并计算** | `[p,!,q,\|,r,&]`<br>`for S in [0, 1 << n)` | -----> | truth table | $O\left(\left(n + m\right) 2^{n}\right)$ |

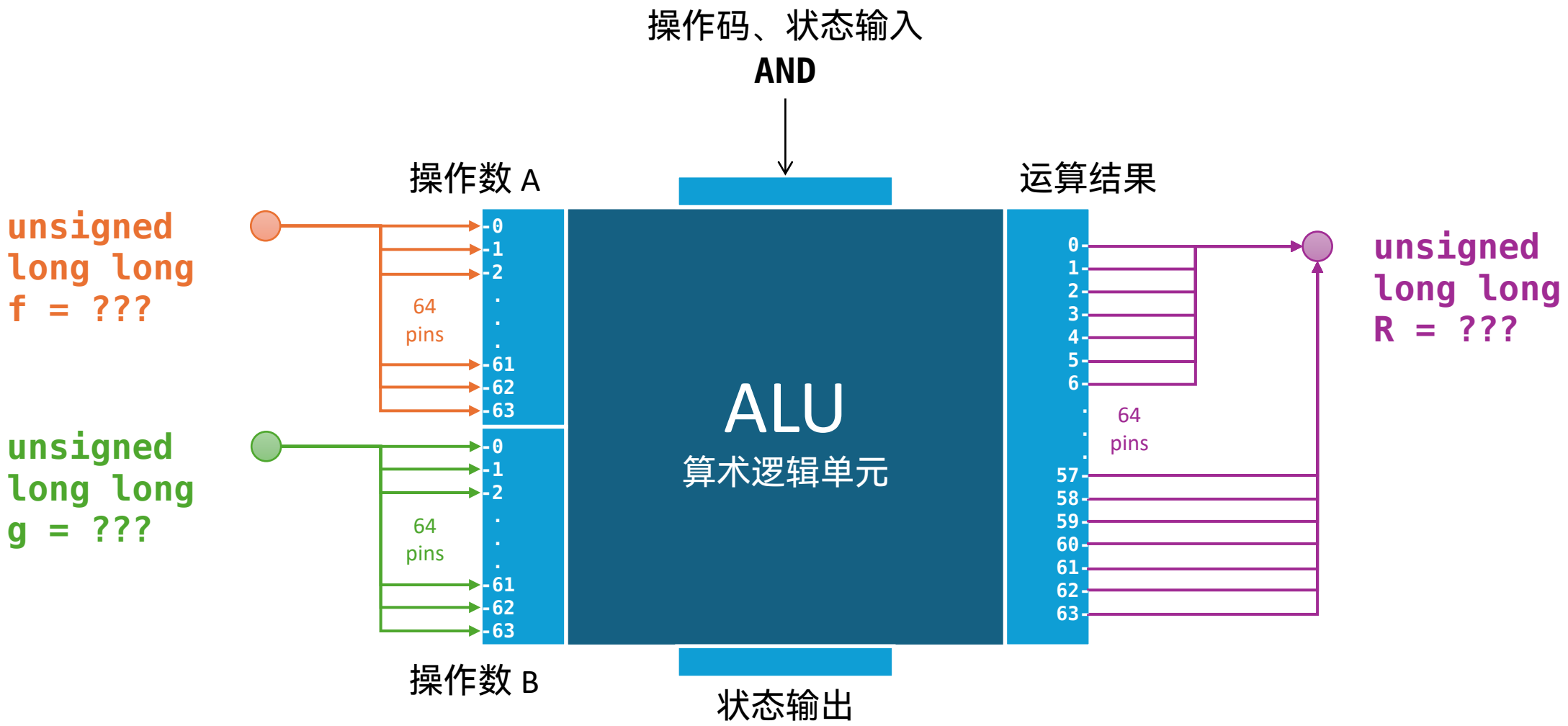
主导项!

时间复杂度 $O\left(\left(n + m\right) 2^{n}\right)$

# 一个周期内的运算 – bool 类型



# 一个周期内的运算 – unsigned long long 类型



运算时间

bool

=

<

unsigned long long

1 bit

承载能力

64 bits

```
  ^^^
```

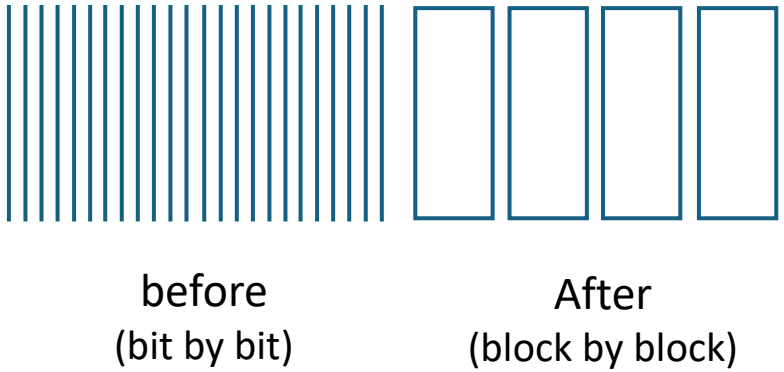
```
FOR block_start FROM 0 TO (2^n - 1) STEP 64:
    FOR EACH variable v IN formula:
        value[v] = compute_64bit_pattern(v, block_start)
    stack = EmptyStack()
    FOR EACH token IN postfix_expression:
        IF token IS variable:
            stack.push( value[token] )
        ELSE IF token IS "!":
            operand = stack.pop()
            stack.push( ~operand )
        ELSE:
            right = stack.pop()
            left = stack.pop()
            IF token == "&":   res = left & right
            IF token == "|":   res = left | right
            IF token == ">":   res = (~left) | right
            IF token == "-":   res = ~(left ^ right)
            stack.push( res )
    final_64bit_result = stack.pop()
    store_to_resultsresult_array(block_start, final_64bit_result)
```

伪代码

**SIMD**（单指令多数据流，Single Instruction, Multiple Data）是一种并行计算技术，允许单个 CPU 指令在同一时钟周期内处理多个数据元素（向量），从而大幅提升多媒体、图形处理、科学计算等数据密集型应用的速度。

dimension: $F^1 \to F^{64} \to F^{256} \to F^{512}$

time: $1 \to 1/64 \to 1/256 \to 1/512$




512 bits

**avx512**, __m512i

256 bits

64 bits

**avx2**, __m256i

1 bit

uint64

bool

# SIMD、“向量化”与 CPU 指令集

对每个命题变元 **var**(i)，即 i = 0,1,2,…,(n-1)

Truth Table

| | | | | |
|---|---|---|---|---|
| **var**(i) | 00000000…00000000 | 11111111…11111111 | 00000000…00000000 | 11111111…11111111 |
| | $2^i$ 个 0 | $2^i$ 个 1 | $2^i$ 个 0 | $2^i$ 个 1 |

| | | |
|---|---|---|
| **var**(i+1) | 0000000000000000…0000000000000000 | 1111111111111111…1111111111111111 |
| | $2^{i+1}$ 个 0 | $2^{i+1}$ 个 1 |

**利用 bit 规律**
**（如 w = 64）**

- 对 **i <= 5**，每个块内的值不随块编号变化 —— **预处理后直接赋值**
- 对 **i >= 6**，每个块内的值全为 0 或全为 1 —— **计算后直接赋值**

**Bit filling** $\boldsymbol{O}\,(\boldsymbol{n}2^{\mathbf{n}}) \rightarrow \boldsymbol{O}\,(\boldsymbol{u}2^{\mathbf{u}} + \boldsymbol{n}2^{\boldsymbol{n}}/\boldsymbol{w}) = \boldsymbol{O}\,(\boldsymbol{w}\mathbf{log}\boldsymbol{w} + \boldsymbol{n}2^{\boldsymbol{n}}/\boldsymbol{w})$

# Benchmark

Timeout = 60s

| Scenario | checker | legacy | recursion | simd_ord_fill | simd_ord_precalc | simd_avx2_fill | simd_avx2_precalc | simd_avx512_precalc | simd_avx512_omp_parallel |
|---|---|---|---|---|---|---|---|---|---|
| var_count = 21, wff_len = 5000 | 0.001951s | 47.129022s | 43.920296s | 0.571434s | 0.541085s | 0.237972s | 0.176265s | 0.118265s | 0.079258s |
| var_count = 22, wff_len = 2000 | 0.002071s | 38.371854s | 34.538539s | 0.530865s | 0.489631s | 0.312511s | 0.193821s | 0.142460s | 0.106942s |
| var_count = 22, wff_len = 120000 | 0.003846s | TIMEOUT | TIMEOUT | 48.236743s | 48.282177s | 12.709980s | 12.622029s | 6.524959s | 0.655331s |
| var_count = 22, wff_len = 500000 | 0.010432s | TIMEOUT | TIMEOUT | TIMEOUT | TIMEOUT | 52.977799s | 53.231869s | 26.935434s | 2.432253s |
| var_count = 22, wff_len = 1000000 | 0.018722s | TIMEOUT | TIMEOUT | TIMEOUT | TIMEOUT | TIMEOUT | TIMEOUT | 54.070732s | 4.844695s |

On WSL2 Ubuntu 22.04.5 LTS, CPU: AMD Ryzen 9 8945HX (16 Cores), RAM: 32 GiB

Compile: `g++ foo.cpp –o foo –O3 –std=c++17 –mavx2 –mavx512f –mavx512bw -fopenmp`

# 总结

递归处理

$$O\left(\mathrm{poly}(n,m)2^n\right)$$

更直接
更具体

性能相似

后缀表达式处理

$$O\left((n+m)2^n\right)$$

更通用
更抽象

一次处理多个

SIMD

$$O\left(n2^{\mathrm{n}}+\frac{m2^n}{w}\right)$$

局部优化

位规律优化

$$\boxed{O\left(\frac{(n+m)2^n}{w}\right)}$$

# Thank you!

**GitHub Link**

**escapist404**/discrete-math-2026-logic-library

- 命题逻辑 python 封装
- SIMD 优化 cpp 实现 benchmark